

Git: Software Engineering Lab Practice

Question & Answer — Local Git Commands Only

Set 1: Repository Initialization, Stashing, and Merging

Q: I. Initialize a new local Git repository in a directory named Nexus-Portal.

```
mkdir Nexus-Portal
cd Nexus-Portal
git init
```

Q: II. Set your username to NexusDev and your email to dev@nexus.com globally.

```
git config --global user.name "NexusDev"
git config --global user.email "dev@nexus.com"
```

Q: III. Create a new file named global layout.css. Stage the file.

```
echo /* Global Layout */ > "global layout.css"
git add "global layout.css"
```

Q: IV. Before you can commit, you receive an urgent request. Use Git to temporarily shelf your uncommitted changes.

```
git stash
Note: If Git reports that there is no initial commit yet, create the initial commit first,
then use git stash.
```

Q: V. Create a new branch named hotfix-sata, switch to it, add a file named sata hdd fix.js, and commit with the required message.

```
git switch -c hotfix-sata
echo // SATA HDD fix > "sata hdd fix.js"
git add "sata hdd fix.js"
git commit -m "Fix SATA HDD external drive read error"
```

Q: VI. Switch back to the master branch and merge the hotfix-sata branch.

```
git switch master
git merge hotfix-sata
```

Q: VII. Retrieve and apply your temporarily stashed changes, then commit global layout.css.

```
git stash pop
git add "global layout.css"
```

```
git commit -m "Add global layout CSS"
```

Set 2: Soft Resets, Reverts, and Branch Management

Q: I. Initialize a new local Git repository named Hospital-System and create an initial commit with a readme.md file.

```
mkdir Hospital-System
cd Hospital-System
git init
echo Hospital System > readme.md
git add readme.md
git commit -m "Initial commit"
```

Q: II. Create two new branches named feature-patient-reg and feature-doctor-select.

```
git branch feature-patient-reg
git branch feature-doctor-select
```

Q: III. Move to feature-patient-reg, add self register.html, and commit with “Add ptiant interface”.

```
git switch feature-patient-reg
echo Self Registration Interface > "self register.html"
git add "self register.html"
git commit -m "Add ptiant interface"
```

Q: IV. Correct the misspelled commit message using a soft reset while keeping files staged, then recommit.

```
git reset --soft HEAD~1
git commit -m "Add patient interface"
```

Q: V. Move to feature-doctor-select, add select logic.js, and commit with “Add doctor selection logic”.

```
git switch feature-doctor-select
echo Doctor Selection Logic > "select logic.js"
git add "select logic.js"
git commit -m "Add doctor selection logic"
```

Q: VI. Revert the commit that added select logic.js without rewriting history.

```
git revert --no-edit HEAD
```

Q: VII. Merge both branches back into master.

```
git switch master
git merge feature-patient-reg
git merge feature-doctor-select
```

Set 3: History Inspection, Hard Resets, and Merge Conflicts

Q: I. Initialize CalculatorApp, create math functions.txt with Base Logic, and make the initial commit.

```
mkdir CalculatorApp
cd CalculatorApp
git init
echo Base Logic > "math functions.txt"
git add "math functions.txt"
git commit -m "Initial commit"
```

Q: II. Create advanced-math and switch to it.

```
git switch -c advanced-math
```

Q: III. Replace the text with Multiplication Logic and commit with “Add multiplication”.

```
notepad "math functions.txt"
Replace the text with:
Multiplication Logic

git add "math functions.txt"
git commit -m "Add multiplication"
```

Q: IV. Replace the text with Divide by Zero Logic and commit with “Add experimental division”.

```
notepad "math functions.txt"
Replace the text with:
Divide by Zero Logic

git add "math functions.txt"
git commit -m "Add experimental division"
```

Q: V. View the commit log and find the exact commit hash for “Add multiplication”.

```
git log --oneline
Find the commit whose message is:
Add multiplication
```

Example:
8d07a64 Add multiplication

Use your actual hash, not the example hash.

Q: VI. Perform a hard reset using the multiplication commit hash.

```
git reset --hard <multiplication-commit-hash>
```

Example:
`git reset --hard 8d07a64`

Q: VII. Switch back to master, replace the text with Addition Logic, and commit with “Add addition”.

```
git switch master
notepad "math functions.txt"
Replace the text with:
Addition Logic
```

```
git add "math functions.txt"
git commit -m "Add addition"
```

Q: VIII. Merge advanced-math into master and trigger the conflict.

```
git merge advanced-math
```

Because both branches changed the same line differently, Git reports a merge conflict.

Q: IX. Resolve the conflict by keeping both addition and multiplication logic, stage the resolved file, and finalize the merge.

```
Open the conflicted file:
notepad "math functions.txt"
```

```
Remove the conflict markers:
<<<<<<< HEAD
=====
>>>>>>> advanced-math
```

```
Keep both:
Addition Logic
Multiplication Logic
```

```
Then:
git add "math functions.txt"
git status
git commit -m "Resolve merge conflict"
```

QUESTION 1

Problem 1: Initialize and Configure Git

Q: 1. Create a new folder named ExamRepo.

```
mkdir ExamRepo
cd ExamRepo
```

Q: 2. Inside the folder, configure Git with your name and email.

```
git config user.name "Your Name"  
git config user.email "your@email.com"
```

Example:

```
git config user.name "NexusDev"  
git config user.email "dev@nexus.com"
```

Q: 3. Initialize the folder as a Git repository.

```
git init
```

Q: 4. Show the current repository status.

```
git status
```

Problem 2: Tracking Files and Creating Commits

Q: 1. Create notes.txt and add the text “Git is a version control system.”

```
echo Git is a version control system. > notes.txt
```

Q: 2. Check the status of the repository.

```
git status
```

Q: 3. Stage the file.

```
git add notes.txt
```

Q: 4. Commit the file with “Added notes.txt”.

```
git commit -m "Added notes.txt"
```

Q: 5. Show the commit history.

```
git log --oneline
```

Problem 3: Modifying Files and Using Reset

Q: 1. Edit notes.txt by adding “Branches allow parallel development.” as a second line.

```
notepad notes.txt
```

```
Add as the second line:  
Branches allow parallel development.
```

Q: 2. Stage and commit this change with a suitable message.

```
git add notes.txt  
git commit -m "Add branch information"
```

Q: 3. View the commit history again.

```
git log --oneline
```

Q: 4. Reset the repository to the first commit.

```
git reset --hard HEAD~1
```

Q: 5. Display notes.txt to show that the second line is removed.

```
type notes.txt
```

Problem 4: Creating and Switching Branches

Q: 1. Create a branch named feature-update.

```
git branch feature-update
```

Q: 2. Switch to feature-update.

```
git switch feature-update
```

Q: 3. Create update.txt containing "This is a feature update."

```
echo This is a feature update. > update.txt
```

Q: 4. Stage and commit the file.

```
git add update.txt  
git commit -m "Add feature update"
```

Q: 5. Show the commit history of the branch.

```
git log --oneline
```

Problem 5: Merging Branches

Q: 1. Switch back to master.

```
git switch master
```

Q: 2. Display repository contents to show update.txt does NOT exist.

```
dir
```

Q: 3. Merge feature-update into master.

```
git merge feature-update
```

Q: 4. Show that update.txt now exists in master.

```
dir
type update.txt
```

Q: 5. Show the updated commit history.

```
git log --oneline --graph --all
```

Problem 6: Creating Another Branch and Simulating Parallel Work

Q: 1. Create a branch named design-change.

```
git branch design-change
```

Q: 2. Switch to design-change.

```
git switch design-change
```

Q: 3. Modify notes.txt by adding “This line was added on the design-change branch.”

```
notepad notes.txt

Add:
This line was added on the design-change branch.
```

Q: 4. Stage and commit the change.

```
git add notes.txt
git commit -m "Add design change"
```

Q: 5. Switch back to master.

```
git switch master
```

Q: 6. Modify notes.txt by adding “This line was added on the main branch.”

```
notepad notes.txt
```

Add:

```
This line was added on the main branch.
```

Q: 7. Stage and commit this change.

```
git add notes.txt  
git commit -m "Add main branch change"
```

Q: 8. Try merging design-change into master and observe what happens.

```
git merge design-change
```

Expected result:

```
CONFLICT (content): Merge conflict in notes.txt
```

Then inspect:

```
git status
```

```
git diff
```

Resolve the conflict in notes.txt by keeping both lines, then:

```
git add notes.txt
```

```
git commit -m "Resolve merge conflict"
```

Local Git Command Reference — Question & Answer

Q: What command shows the Git version?

```
git --version
```

Q: What command opens Git help?

```
git help
```

Q: How do you get help for a specific command?

```
git help <command>
```


Q: How do you show short help?

```
git <command> -h
```

Q: How do you list all configuration?

```
git config --list
```

Q: How do you show global configuration?

```
git config --global --list
```

Q: How do you show local repository configuration?

```
git config --local --list
```

Q: How do you show the configured username?

```
git config user.name
```

Q: How do you show the configured email?

```
git config user.email
```

Q: How do you set a global username?

```
git config --global user.name "Name"
```

Q: How do you set a global email?

```
git config --global user.email "Mail"
```

Q: How do you set a repository-specific username?

```
git config user.name "Name"
```

Q: How do you set a repository-specific email?

```
git config user.email "Mail"
```

Q: How do you initialize a repository?

```
git init
```

Q: How do you initialize a specific folder?

```
git init <folder>
```

Q: How do you clone a local repository?

```
git clone <path>
```

Q: How do you check repository status?

```
git status
```

Q: How do you show short status?

```
git status -s
```

Q: How do you show the repository root?

```
git rev-parse --show-toplevel
```

Q: How do you check whether you are inside a Git work tree?

```
git rev-parse --is-inside-work-tree
```

Q: How do you stage a file?

```
git add <file>
```

Q: How do you stage multiple files?

```
git add <file1> <file2>
```

Q: How do you stage all changes?

```
git add . OR git add -A
```

Q: How do you unstage a file?

```
git restore --staged <file>
```

Q: How do you unstage everything?

```
git restore --staged .
```

Q: How do you create a commit?

```
git commit -m "message"
```

Q: How do you commit tracked changes with a message?

```
git commit -am "message"
```

Q: How do you amend the last commit message?

```
git commit --amend -m "message"
```

Q: How do you compare unstaged changes?

```
git diff
```

Q: How do you compare staged changes?

```
git diff --cached
```

Q: How do you compare all changes with HEAD?

```
git diff HEAD
```

Q: How do you view full commit history?

```
git log
```

Q: How do you view one-line history?

```
git log --oneline
```

Q: How do you view a graph of all branches?

```
git log --oneline --graph --all
```

Q: How do you show a specific commit?

```
git show <commit>
```

Q: How do you create a branch?

```
git branch <name>
```

Q: How do you list branches?

```
git branch
```

Q: How do you delete a merged branch?

```
git branch -d <name>
```

Q: How do you force-delete a branch?

```
git branch -D <name>
```

Q: How do you switch branches?

```
git switch <branch>
```

Q: How do you create and switch to a branch?

```
git switch -c <branch>
```

Q: How do you merge a branch?

```
git merge <branch>
```

Q: How do you abort an unfinished merge?

```
git merge --abort
```

Q: How do you continue a merge after resolving conflicts?

```
git merge --continue
```

Q: How do you soft-reset one commit?

```
git reset --soft HEAD~1
```

Q: How do you mixed-reset one commit?

```
git reset --mixed HEAD~1
```

Q: How do you hard-reset one commit?

```
git reset --hard HEAD~1
```

Q: What does soft reset do?

Moves HEAD while keeping changes staged.

Q: What does mixed reset do?

Moves HEAD while keeping changes in the working tree unstaged.

Q: What does hard reset do?

Moves HEAD and discards tracked working-tree/index changes.

Q: How do you revert the latest commit?

```
git revert HEAD
```

Q: How do you revert a specific commit?

```
git revert <commit-id>
```

Q: What is the difference between reset and revert?

Reset rewrites the current branch history; revert creates a new commit that reverses an earlier commit.

Q: How do you stash current changes?

```
git stash
```

Q: How do you include untracked files in a stash?

```
git stash -u
```

Q: How do you list stashes?

```
git stash list
```

Q: How do you apply the latest stash and remove it?

```
git stash pop
```

Q: How do you apply a stash without removing it?

```
git stash apply
```

Q: How do you create a tag?

```
git tag <name>
```

Q: How do you list tags?

```
git tag
```

Q: How do you apply a commit to the current branch?

```
git cherry-pick <commit>
```

Q: How do you show HEAD movements for recovery?

```
git reflog
```

Q: How do you list tracked files?

```
git ls-files
```

Q: How do you remove a tracked file and stage the deletion?

```
git rm <file>
```

Q: How do you stop tracking a file but keep it locally?

```
git rm --cached <file>
```

Q: How do you preview untracked-file deletion?

```
git clean -n
```

Q: How do you delete untracked files?

```
git clean -f
```

Q: How do you create ignored-file rules?

Create a .gitignore file.

Q: How do you check why a file is ignored?

```
git check-ignore -v <file>
```

Q: How do you show who changed each line?

```
git blame <file>
```

Q: How do you search tracked files for text?

```
git grep "text"
```

Q: How do you start a bisect session?

```
git bisect start
```

Q: How do you mark the current commit as bad?

```
git bisect bad
```

Q: How do you mark a known commit as good?

```
git bisect good <commit>
```

Q: How do you finish bisect?

```
git bisect reset
```

Q: How do you list worktrees?

```
git worktree list
```

Q: How do you add a worktree?

```
git worktree add <path> <branch>
```

Q: How do you add a local submodule?

```
git submodule add <path>
```

Q: How do you initialize submodules?

```
git submodule update --init
```

Q: How do you list notes?

```
git notes list
```

Q: How do you optimize/clean the repository?

```
git gc
```

Q: How do you check repository integrity?

```
git fsck
```

Q: How do you count Git objects?

```
git count-objects
```

Q: How do you show an object's type?

```
git cat-file -t <object>
```

Q: How do you show an object's contents?

```
git cat-file -p <object>
```

Q: How do you create a hash from a file?

```
git hash-object <file>
```

Q: How do you show all references?

```
git show-ref
```

Q: How do you get the current commit?

```
git rev-parse HEAD
```

Q: How do you create a ZIP archive of HEAD?

```
git archive --format=zip HEAD
```

Q: How do you apply a patch?

```
git apply <patch>
```

Q: How do you create format-patch files?

```
git format-patch -1
```


Q: How do you create a Git alias?

```
git config --global alias.st status
```

Local Git Scope

Excluded remote/GitHub workflow commands: git push, git pull, git fetch, git remote, git ls-remote, and cloning from remote URLs.

